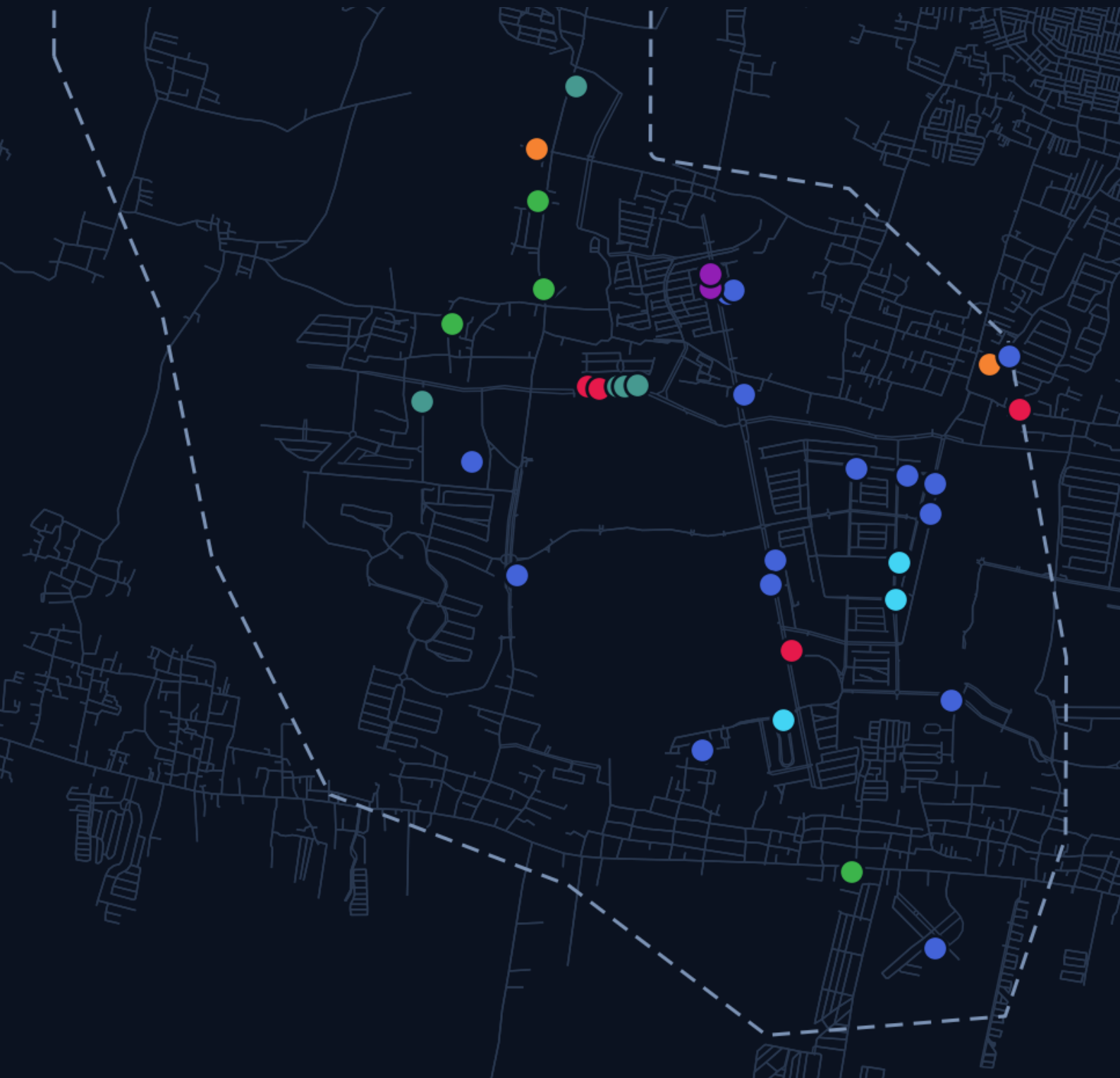
PANDUAN BELAJAR

Citraland Sport Finder

Cara kerja aplikasi GIS pencari tempat olahraga terdekat



Computer Graphics · Informatika · Universitas Ciputra

47 tempat olahraga · 27 km²

Daftar isi

1. Gambaran umum 3

2. Konsep GIS yang dipakai 5

3. Alur data dari klik sampai peta 8

4. Penjelasan tiap file 9

5. Menjalankan aplikasi 11

6. Soal latihan 12

7. Kunci jawaban 13

Cara memakai panduan ini

- Baca Bab 1 dan 2 dulu untuk memahami idenya, lalu buka kodenya bersama Bab 3 dan 4.
- Coba jalankan aplikasinya (Bab 5) sebelum mengerjakan soal, supaya tahu perilakunya.
- Kerjakan soal di Bab 6 tanpa melihat kunci. Kuncinya ada di bab setelahnya.
- Semua angka contoh di panduan ini dihitung dari data dan kode proyek yang sebenarnya.

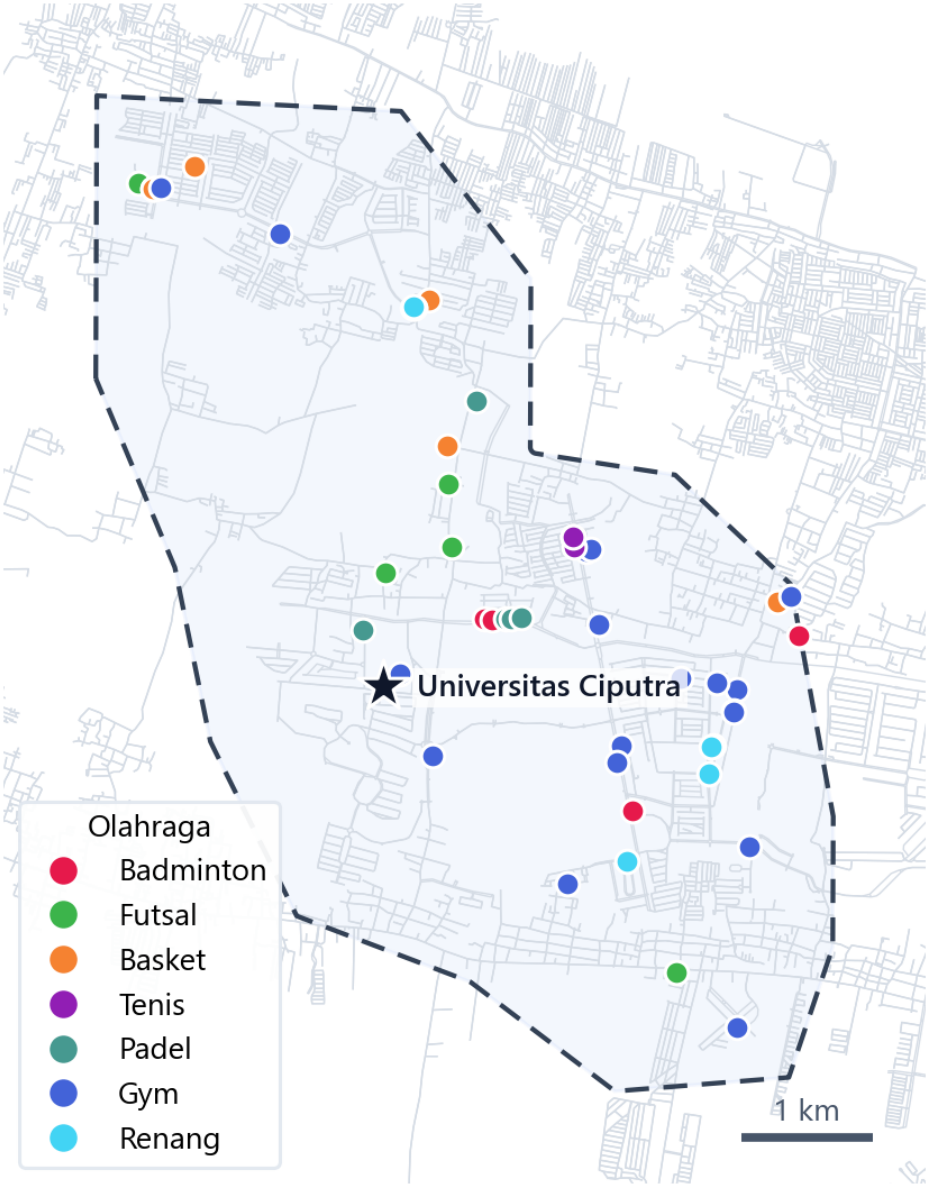
BAB 1

Gambaran umum

Aplikasi ini menjawab satu pertanyaan: tempat olahraga mana di CitraLand yang paling dekat dari alamatku, kalau lewat jalan?

Pengguna memilih jenis olahraga dan mengetik alamat awal. Aplikasi menggambar rute jalan ke tempat terpilih di peta dan menampilkan jaraknya dalam kilometer. Semuanya dibangun dari dua **layer** data spasial:

- **Layer 1, titik:** tempat olahraga dari Google Places, disimpan di `data/venues.geojson`.
- **Layer 2, poligon:** batas kawasan CitraLand, disimpan di `data/citraland_area.geojson`.



Gambar 1. Kedua layer di atas jaringan jalan OSM. Garis putus-putus adalah batas kawasan, bintang adalah titik awal bawaan (Universitas Ciputra). Warna titik sama dengan warna marker di aplikasi.

Empat langkah GIS

Setiap kali tombol **Tampilkan Rute** ditekan, data melewati empat langkah. Contoh ini memakai olahraga basket dari Universitas Ciputra. Kolom kanan menunjukkan jumlah baris yang tersisa.

–	**Data awal** Semua hasil Google Places di data/venues.geojson	**82** baris
1	**spatial_join** Simpan titik yang berada di dalam poligon CitraLand	**52** baris, 47 tempat
2	**filter_by_sport** Saring berdasarkan kolom Jenis_Olahraga	**7** baris basket
3	**rank_by_distance** Hitung jarak garis lurus dalam meter, urutkan, ambil 3 teratas	**3** kandidat
4	**route_to_nearest** Hitung rute jalan ke tiap kandidat, pilih yang terpendek	**1** hasil

Kenapa 52 baris tetapi hanya 47 tempat?
Satu tempat bisa muncul untuk beberapa olahraga. Citraland Square Park, misalnya, tercatat sebagai basket dan tenis. Satu baris mewakili pasangan (tempat, olahraga).

Teknologi yang dipakai

Library	Tugasnya di proyek ini
GeoPandas, Shapely	Membaca GeoJSON, mengubah CRS, spatial join, menghitung jarak, geometri titik dan garis
OSMnx	Mengunduh jaringan jalan OpenStreetMap dan mencari rute terpendek
Folium	Membuat peta interaktif Leaflet sebagai HTML
Flask	Server web dengan satu halaman
Requests, Google API	Mengambil tempat dari Places API dan mengubah alamat menjadi koordinat

BAB 2

Konsep GIS yang dipakai

Empat konsep ini adalah inti proyek. Kalau keempatnya paham, sisanya tinggal membaca kode.

2.1 Sistem koordinat (CRS)

Koordinat GPS dan Google berupa derajat lintang dan bujur. Menghitung jarak langsung dari angka ini menghasilkan derajat, bukan meter, dan panjang satu derajat bujur berubah menurut posisi di bumi. Karena itu aplikasi memakai dua CRS:

CRS	Kode	Dipakai untuk
WGS 84	EPSG:4326	Input dari Google, file GeoJSON, dan tampilan peta Folium
WGS 84 / UTM zona 49S	EPSG:32749	Semua perhitungan jarak. Satuannya meter dan zona ini mencakup Surabaya

> **Aturan di kode**
> Data dibaca lalu langsung diubah ke UTM dengan `.to_crs(config.CRS_UTM)`, dan baru dikembalikan ke 4326 saat digambar. `rank_by_distance` melempar `ValueError` kalau layer yang masuk bukan UTM, dan `test_distances_use_utm` memastikan penjagaan ini tetap ada.

2.2 Spatial join

Pencarian Google bisa mengembalikan tempat di luar CitraLand. Spatial join menyaring titik berdasarkan hubungan ruangnya dengan poligon, bukan berdasarkan kolom yang sama:

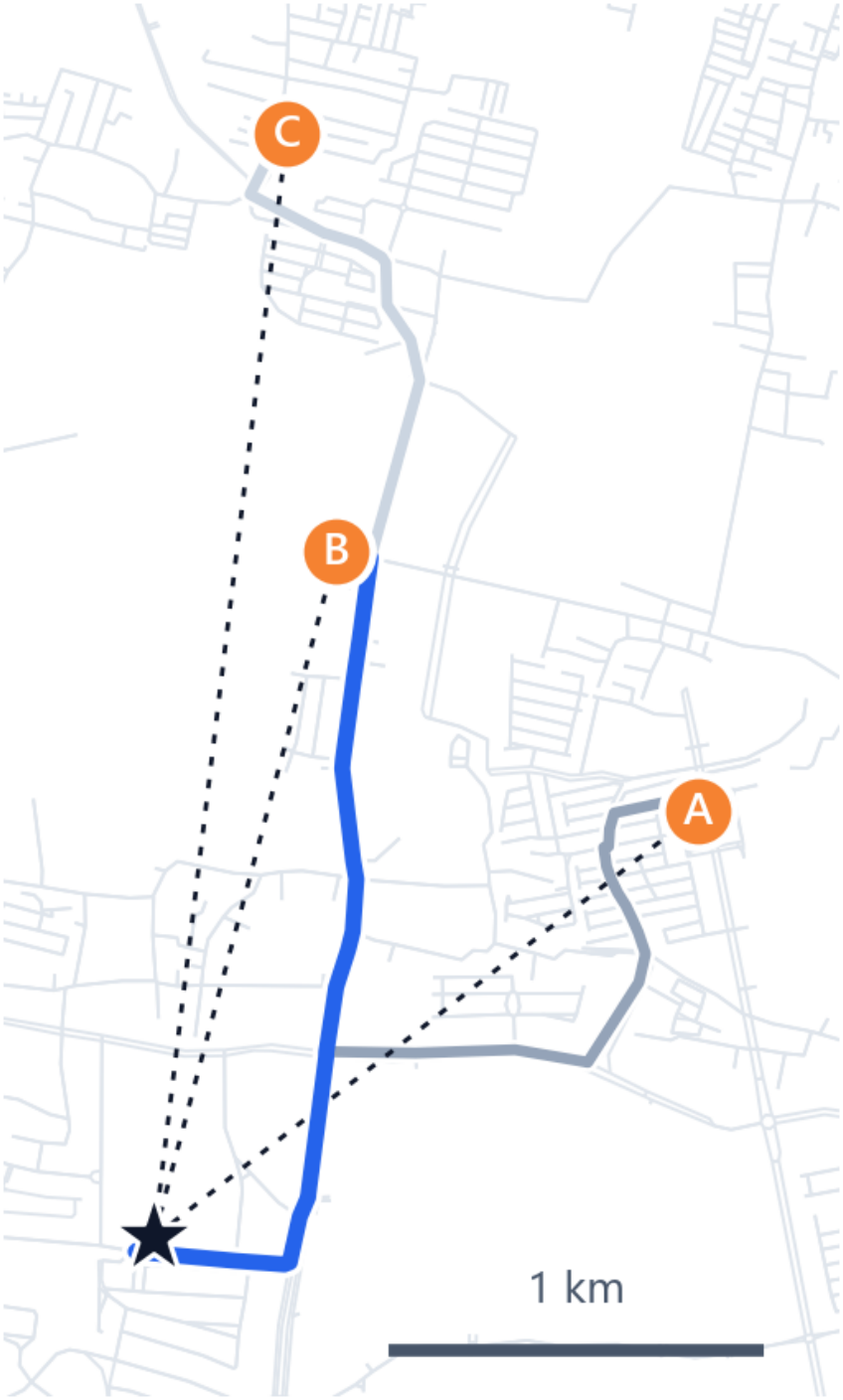
gis.py · spatial_join

```
joined = gpd.sjoin(venues, boundary[["geometry"]], how="inner",
                   predicate="intersects")
return joined.drop(columns="index_right").drop_duplicates(
    subset=["Tempat_ID", "Jenis_Olahraga"])
```

- `predicate="intersects"`: titik tepat di tepi poligon ikut masuk.
- `how="inner"`: hanya baris yang cocok yang disimpan. Di sini 82 baris menjadi 52.
- `index_right`: kolom tambahan dari sjoin yang tidak diperlukan, jadi dibuang.

2.3 Garis lurus vs jalan

Tempat terdekat secara garis lurus belum tentu terdekat lewat jalan. Menghitung rute ke semua tempat juga mahal, jadi aplikasi bekerja dua tahap: urutkan semua berdasarkan garis lurus, lalu hitung rute hanya untuk `ROUTE_CANDIDATES = 3` teratas. Basket adalah contoh nyatanya:



Gambar 2. Rute dari Universitas Ciputra ke tiga kandidat basket. Garis titik-titik adalah jarak lurus, garis biru adalah rute terpilih menuju B.

	Tempat	Garis lurus	Lewat jalan
A	Citraland Square Park	1.842 m (urutan 1)	2.735 m
B	**Citraland Cluster Taman Puspa Raya**	1.891 m (urutan 2)	**2.379 m (terpilih)**
C	Lapangan Kendung	2.961 m (urutan 3)	3.798 m

> **Yang perlu diingat**
> A menang 49 m secara garis lurus, tetapi B lebih pendek 356 m lewat jalan. Kalau aplikasi hanya merutekan kandidat pertama, jawabannya salah.

2.4 Routing di graf jalan

OSMnx mengunduh jaringan jalan sebagai **graf**: persimpangan dan ujung jalan menjadi *node*, ruas jalan menjadi *edge* dengan atribut `length`. Graf proyek ini punya 10.262 node dan 25.090 edge. Satu ruas dua arah dihitung dua edge.

```
gis.py · _route

o_node = ox.distance.nearest_nodes(graph, X=origin.x, Y=origin.y)
d_node = ox.distance.nearest_nodes(graph, X=dest.x, Y=dest.y)
nodes = ox.routing.shortest_path(graph, o_node, d_node, weight="length")
```

- **Snap:** titik awal dan tujuan jarang tepat di atas jalan, jadi keduanya dipindahkan ke node terdekat.
- `weight="length"`: yang diminimalkan adalah panjang jalan, bukan waktu tempuh.
- **Buffer 2.500 m:** graf mencakup area di luar batas supaya pengguna dari Lontar atau Sambikerep tetap bisa dirutekan masuk.
- **Cache:** unduhan pertama butuh 1 sampai 2 menit, lalu tersimpan di `data/graph.graphml` dan di memori.

> **Akibat dari snap**
> Jarak yang ditampilkan hanya menjumlahkan edge antar-node. Potongan dari titik asli ke node terdekat digambar di peta tetapi tidak dihitung. Dua tempat yang snap ke node yang sama juga mendapat jarak yang sama: WESTON SPORTS dan Sekolah Badminton sama-sama 1.777 m.

BAB 3

Alur data dari klik sampai peta

Satu request menelusuri tiga file. Ikuti alur ini sambil membuka kodenya.

1	**app.py · index()** Baca parameter URL: action, sport, address	**Flask**
2	**gis.spatial_join** Venue di dalam poligon CitraLand	**GIS**
3	**gis.filter_by_sport** Hanya olahraga yang dipilih	**GIS**
4	**app.py · resolve_origin** Alamat menjadi lat, lon lewat Google, atau fallback ke Universitas Ciputra	**Google**
5	**gis.rank_by_distance** Urutkan berdasarkan jarak garis lurus dalam meter	**GIS**
6	**gis.route_to_nearest** Rute jalan ke 3 teratas, pilih yang terpendek	**OSMnx**
7	**app.py · build_map** Gambar batas, titik venue, marker alamat, dan garis rute	**Folium**
8	**templates/index.html** Peta dan panel hasil: nama, olahraga, jarak km	**Jinja**

Tiga mode halaman

Parameter `action` di URL menentukan seberapa jauh alur di atas berjalan. Langkah 4 sampai 6 hanya berjalan pada mode `route`.

action	Kapan terjadi	Hasil
`(kosong)`	Halaman pertama kali dibuka	Semua tempat, tanpa filter dan tanpa rute
`filter`	Dropdown olahraga diganti	Titik difilter, rute tidak dihitung
`route`	Tombol Tampilkan Rute ditekan	Titik difilter, rute dan kartu hasil tampil

> **Kenapa dropdown tidak langsung menghitung rute?**
> Karena routing lebih lambat. JavaScript di `index.html` mengubah `action` menjadi `filter` saat dropdown berubah, sehingga titik di peta langsung tersaring tanpa menghitung rute.

BAB 4

Penjelasan tiap file

Peta kode proyek, disusun dari yang paling penting untuk dibaca.

File	Peran
`gis.py`	Inti GIS: memuat layer, spatial join, filter, ranking, graf jalan, routing
`app.py`	Aplikasi Flask dengan satu route, membangun peta Folium, mengirim data ke template
`data_fetch.py`	Mengambil tempat dari Places API, geocoding alamat, membangun poligon batas
`config.py`	Konstanta bersama: path, kode CRS, API key, titik fallback, kamus SPORTS
`templates/index.html`	Peta di belakang dan panel di depan: form dan kartu hasil
`static/style.css`	Tampilan panel dan layout responsif (panel kiri di desktop, bottom sheet di ponsel)
`tests/test_gis.py`	14 tes pytest: spatial join, filter, urutan jarak, UTM, filter nama, batas area
`data/*.geojson`	venues (data asli), venues_sample (8 contoh), citraland_area (batas yang dipakai), citraland_osm dan citraland_manual (cadangan)

4.1 gis.py, fungsi per fungsi

Fungsi	Masukan	Keluaran
`load_boundary`	–	Poligon dalam UTM. Urutan sumber: citraland_area, cache OSM, unduh OSM, poligon manual
`load_venues`	–	Titik dalam UTM: cache Google Places, kalau tidak ada data contoh
`point_from_latlon`	lat, lon	GeoSeries satu titik pengguna dalam UTM
`spatial_join`	venues, boundary	Venue di dalam atau di tepi poligon, tanpa duplikat
`filter_by_sport`	venues, sport	Semua kalau "semua", selain itu baris olahraga tersebut
`rank_by_distance`	venues, titik	Tambah kolom Jarak_m, urut naik
`get_graph`	boundary	Graf jalan UTM, di-cache di disk dan memori
`_route`	graf, awal, tujuan	Dict length_m dan line, atau None kalau tidak ada rute
`route_to_nearest`	ranked, titik, graf	Tabel dengan Jarak_Rute_m dan rute terbaik, garisnya sudah dalam 4326

4.2 config.py, kamus SPORTS

Satu kamus mengatur dropdown, warna marker, kata kunci pencarian, dan filter nama sekaligus. Menambah olahraga cukup menambah satu entri, lalu mengambil ulang data.

```
config.py

"tenis": {"label": "Tenis", "query": "lapangan tenis", "color": "#911eb4",
          "words": ["tenis", "tennis"], "not_words": ["tenis meja", "table tennis"]},
```

Kunci	Fungsi
`label`	Teks di dropdown dan legenda
`query`	Teks pencarian ke Places API, ditambah "CitraLand Surabaya"
`color`	Warna marker di peta
`words`	Kata di nama tempat yang menandakan olahraga ini
`not_words`	Kata yang menyingkirkan olahraga ini walau ada kata yang cocok

4.3 data_fetch.py

- `search_places`: satu Text Search per olahraga dengan *location bias* berupa lingkaran radius 6 km di pusat CitraLand.
- `names_other_sport`: Text Search itu longgar, "lapangan futsal" juga mengembalikan lapangan tenis. Fungsi ini membuang nama yang jelas milik olahraga lain, tetapi menyimpan nama tanpa olahraga seperti "Citraland Square Park".
- `geocode`: mencoba Geocoding API. Kalau ditolak (REQUEST_DENIED), beralih ke Places Text Search.
- `build_area`: menggabungkan dua poligon OSM dengan poligon gambar tangan, lalu `buffer(50).buffer(-50)` menutup celah tipis dan lubang di dalamnya dibuang.

4.4 app.py

- `BOUNDARY`, `VENUES_ALL`: dimuat sekali saat server menyala, bukan setiap request.
- `resolve_origin`: tanpa API key atau saat geocoding gagal, aplikasi memakai Universitas Ciputra dan menampilkan peringatan.
- `build_map`: rute digambar dua kali, garis putih tebal di bawah garis gelap, supaya tetap terbaca di atas jalan dan marker.
- `fitBounds`: zoom dilakukan di browser karena padding bergantung pada lebar layar, yang tidak diketahui Python.
- `km_text`: mengubah meter menjadi teks dengan koma desimal, misalnya 2,38 km.

BAB 5

Menjalankan aplikasi

Repo sudah menyertakan data tempat, jadi aplikasi bisa langsung jalan tanpa API key.

```
Command Prompt di Windows

py -3.12 -m venv .venv
.venv\Scripts\pip install -r requirements.txt
copy .env.example .env
.venv\Scripts\flask --app app run
```

Lalu buka `http://127.0.0.1:5000`. Rute pertama butuh internet dan 1 sampai 2 menit untuk mengunduh graf jalan.

Perintah atau langkah	Kegunaan
`Isi GOOGLE_MAPS_API_KEY di .env`	Opsional. Membuat alamat bisa dicari. Aktifkan Places API (New) dan Geocoding API. Tanpa key, titik awal selalu Universitas Ciputra
`python data_fetch.py --refresh`	Mengambil ulang data tempat (butuh key)
`python data_fetch.py --boundary`	Membangun ulang poligon batas
`.venv\Scripts\python -m pytest`	Menjalankan 14 tes

Batasan yang perlu diketahui

- **Batas kawasan sebagian perkiraan.** Northwest, The GreenLake, dan celah antar bagian digambar tangan karena OSM tidak punya poligonnya. Luas totalnya 27 km².
- **Tenis meja belum punya data.** Olahraga ini ada di dropdown, tetapi tidak ada satu pun tempatnya di dalam batas, jadi hasilnya kosong.
- **Lokasi Google tidak selalu sesuai label.** Private Fitness berlabel "Ngagel", tetapi koordinatnya masuk poligon Citraland Utara sehingga dihitung CitraLand.
- **Jarak, bukan waktu.** Aplikasi tidak memperkirakan waktu tempuh dan tidak memperhitungkan kemacetan.
- **Jam buka tidak dipakai.** Datanya masih disimpan di GeoJSON, tetapi filter buka sudah dihapus dari kode.

BAB 6

Soal latihan

Jawab dengan kata-katamu sendiri, lalu cocokkan di Bab 7.

1. Mengapa jarak dihitung di EPSG:32749, bukan EPSG:4326?

2. Apa beda predicate "intersects" dan "within" untuk titik di tepi poligon?

3. Pada contoh basket, mengapa B yang terpilih padahal A lebih dekat secara garis lurus?

4. Mengapa rute dihitung ke 3 kandidat, bukan 1 atau semua?

5. Mengapa WESTON SPORTS dan Sekolah Badminton punya jarak rute yang sama?

6. Apa fungsi names_other_sport?

7. Apa yang terjadi kalau API key kosong?

8. Mengapa graf jalan diberi buffer 2.500 m?

9. Mengapa fitBounds dijalankan di browser, bukan di Python?

10. Bagaimana menambah olahraga baru?

Tantangan di kode

- Ubah `ROUTE_CANDIDATES` menjadi 1, lalu cari rute basket dari titik awal bawaan. Tempat mana yang sekarang terpilih?
- Ganti warna badminton di `config.SPORTS` dan lihat apa saja yang ikut berubah di halaman.
- Tulis tes baru untuk `filter_by_sport` dengan olahraga yang tidak ada di data. Apa yang seharusnya dikembalikan?

BAB 7

Kunci jawaban

Jawaban singkat untuk soal Bab 6, ditambah glosarium.

1. EPSG:4326 memakai derajat, sehingga hasilnya bukan meter. EPSG:32749 (UTM 49S) memakai meter dan cocok untuk wilayah Surabaya.
2. "intersects" menyertakan titik di tepi. "within" mengharuskan titik berada di bagian dalam, jadi titik tepat di tepi terbuang.
3. Aplikasi memilih rute jalan terpendek di antara 3 kandidat. B 2.379 m lewat jalan, A 2.735 m.
4. Satu kandidat bisa memberi jawaban salah seperti contoh basket. Semua kandidat lebih lambat. Tiga adalah kompromi, tetapi tidak menjamin hasil terbaik secara mutlak: tempat di urutan 4 tidak pernah diperiksa.
5. Keduanya di-snap ke node jalan yang sama, dan jarak dihitung antar-node.
6. Membuang hasil Places yang namanya jelas milik olahraga lain, sambil menyimpan nama yang tidak menyebut olahraga.
7. Data tempat tetap dibaca dari file, dan titik awal memakai Universitas Ciputra dengan peringatan di panel.
8. Agar pengguna di luar CitraLand, misalnya Lontar atau Sambikerep, tetap punya jalan menuju kawasan.
9. Padding bergantung pada ukuran layar: panel di kiri pada desktop, bottom sheet pada ponsel. Python tidak mengetahui ukuran layar.
10. Tambah satu entri di config.SPORTS, lalu jalankan data_fetch.py --refresh. Dropdown dan legenda terisi otomatis.

Tantangan di kode

- Citraland Square Park (A), dengan jarak 2.735 m. Ini bukan yang terpendek lewat jalan.
- Warna marker dan legenda ikut berubah, karena keduanya membaca config.SPORTS.
- GeoDataFrame kosong. Di mode route, app.py menanganinya dengan pesan "Tidak ada tempat yang cocok dengan filter ini."

Glosarium

Istilah	Arti
Layer	Satu lapisan data spasial, misalnya titik tempat atau poligon batas
GeoJSON	Format teks untuk menyimpan geometri beserta atributnya
GeoDataFrame	Tabel pandas dengan kolom geometri, dari GeoPandas
CRS	Coordinate Reference System, aturan yang mengaitkan angka koordinat dengan posisi di bumi
Spatial join	Menggabungkan atau menyaring layer berdasarkan hubungan ruang
Node, edge	Titik dan ruas dalam graf jaringan jalan
Snap	Memindahkan titik ke objek terdekat, di sini node jalan
Geocoding	Mengubah alamat teks menjadi koordinat
Location bias	Petunjuk agar hasil Places diprioritaskan di sekitar lokasi tertentu